ASSIGNMENT 2 – CO2

STRINGS AND COLLECTIONS

Name: DEVALAKSHMI. S

Register Number: 192511134

GitHub Repository: GitHub - 192511134simats/CSA09-192511134-Assignment1 at Assignment-2

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? (

1. Problem understanding and prediction

A railway e-ticket combines fields such as PNR, passenger name, train number, coach, berth, fare and journey date. The ticket is generated repeatedly, so the choice of text-building class can affect execution time and temporary memory use.

String is immutable. Once a String object is created, its contents cannot be changed. An expression such as text

= text + field creates a new String containing the old text and the new field. The old intermediate value becomes eligible for garbage collection when it is no longer referenced. Repeating this operation copies an increasingly large prefix many times.

StringBuilder is mutable. It keeps a resizable character buffer and appends fields to that buffer. It normally avoids creating a new complete String after every append. It is not synchronized, so it is the preferred choice for building a local ticket inside one thread.

StringBuffer is also mutable, but its public methods are synchronized. This makes individual operations thread-safe when the same buffer is shared correctly. Synchronization adds overhead. For a buffer used only by one thread, StringBuffer is usually slower than StringBuilder.

Approach	Prediction for many fields	Reason
Repeated String +	Slowest as fields increase	Creates repeated intermediate strings and recopies existing characters.
StringBuilder	Usually fastest for one ticket	Appends into a mutable buffer; amortized linear work.
StringBuffer	Similar pattern but some extra overhead	Synchronized methods protect individual operations.

1. Three workloads

Workload	Tickets	Fields per ticket	Purpose
A – Small	1	6	Represents a single short ticket.
B – High load	10,000	50	Shows repeated allocation and copying under load.
C – Larger stress test	50,000	100	Shows scaling when both ticket count and field count grow.

Use identical field values and the same output requirement for all three approaches. Do not print every ticket during timing because console output would dominate the measurement.

1. Java implementation

```
import java.util.*;



public class TicketBenchmark {

static String[] fields(int count) { String[] a = new String[count]; for (int i = 0; i < count; i++)

a[i] = "FIELD" + i + "-PNR123456789";

return a;
```

```
}

static String usingString(String[] fields) { String result = "";
for (String field : fields) result = result + field + "|";
return result;
}

static String usingBuilder(String[] fields) { StringBuilder result = new StringBuilder();
for (String field : fields) result.append(field).append('|'); return result.toString();
}

static String usingBuffer(String[] fields) { StringBuffer result = new StringBuffer();
for (String field : fields) result.append(field).append('|'); return result.toString();
}

static long run(String method, int tickets, int fieldCount) { String[] data = fields(fieldCount);
long start = System.nanoTime(); long checksum = 0;
for (int i = 0; i < tickets; i++) { String s;
if (method.equals("String")) s = usingString(data);
else if (method.equals("Builder")) s = usingBuilder(data); else s = usingBuffer(data);
checksum += s.length(); // prevents the result from being unused
}
long elapsed = System.nanoTime() - start; System.out.println(method + " checksum=" + checksum); return elapsed;
}

public static void main(String[] args) {
int[][] tests = {{1, 6}, {10000, 50}, {50000, 100}};
for (int[] test : tests) { System.out.println("\\nTickets=" + test[0] +
", fields=" + test[1]); for (int warm = 0; warm < 2; warm++) {
run("String", test[0], test[1]);
```

```
run("Builder", test[0], test[1]);

run("Buffer", test[0], test[1]);

}

// Record several measured repetitions after warm-up. for (int repeat = 1; repeat <= 5; repeat++) {

System.out.println("Measured repetition " + repeat); System.out.println("String ns=" +

run("String", test[0], test[1])); System.out.println("Builder ns=" +

run("Builder", test[0], test[1])); System.out.println("Buffer ns=" +

run("Buffer", test[0], test[1]));

}

}

}

}
```

1. Complexity and memory analysis

Let n be the number of fields and L be the average number of characters appended per field. With repeated concatenation, the first field is copied once, the second operation copies the previous content plus the new field, and later operations copy progressively larger prefixes. The total copied characters are proportional to L(1 + 2 +

... + n), which is $O(Ln^2)$, or $O(n^2)$ when field length is treated as constant.

With StringBuilder, appending all fields takes O(Ln) total character work, conventionally written O(n) when field size is fixed. The final String creation requires one additional copy of the final buffer, so total work remains linear. StringBuffer has the same broad asymptotic complexity as StringBuilder, with synchronization overhead.

Measure	**Repeated +**	**StringBuilder / StringBuffer**
Time for n fixed-size fields	$O(n^2)$ character copying	O(n) amortized append work
Final result space	O(n)	O(n)
Temporary objects	Many intermediate String objects	Fewer intermediate full strings; buffer may resize
Thread safety	String object itself is immutable	Builder not synchronized; Buffer synchronized

1. Recording the experiment

Workload / method	Run 1 (ns)	Run 2 (ns)	Run 3 (ns)	Run 4 (ns)	Run 5 (ns)	Average (ns)
A – String						
A – Builder						
A – Buffer						
B – String						
B – Builder						
B – Buffer						
C – String						
C – Builder						
C – Buffer						

Estimate intermediate String creation for repeated concatenation as roughly one new result String for each concatenation operation (the exact number depends on compiler/JVM optimisations). StringBuilder and StringBuffer create the mutable buffer and may allocate larger buffers as capacity grows; toString() creates the final immutable String. Heap usage measured with Runtime.freeMemory() is noisy and affected by garbage collection, so treat it as an approximate observation rather than an exact allocation count.

1. Multithreaded design decision

For each individual ticket, use a new local StringBuilder. Each thread owns its builder, so there is no shared mutable state and synchronization is unnecessary. For a shared booking log, do not assume that simply using one StringBuffer solves the complete logging problem: synchronized append protects individual calls, but a multi-step log record may still need to be written atomically. A thread-safe logging framework or a synchronized logging method is better. If the assignment

requires choosing between the two classes, StringBuffer is safer for a buffer genuinely shared by multiple threads; StringBuilder is preferred for per-ticket text.

1. Alternative: String.join, StringJoiner and String.format

String.join(delimiter, fields) is concise when the fields are already available as strings. StringJoiner is useful when building a delimited sequence incrementally and supports prefix/suffix. String.format is readable for formatted output, but format parsing and formatting overhead can make it slower for large repeated workloads.

```
String ticket1 = String.join(" | ", fields);
```

```
StringJoiner joiner = new StringJoiner(" | ", "TICKET[", "]"); for (String field : fields) joiner.add(field);
```

```
String ticket2 = joiner.toString();
```

```
String ticket3 = String.format("PNR=%s | NAME=%s",
```

```
pnr, passengerName);
```

The recommended solution is StringBuilder for repeated per-ticket appends. String.join is a good alternative for an already prepared array or list. String.format is most appropriate when readability and formatting features are more important than maximum throughput.

Q2 – When Is ArrayList an Appropriate Engineering Choice?

1. Comparison and experiment design

ArrayList stores elements in a resizable array. It provides fast indexed access because the k-th element can be reached directly. Adding at the end is amortized O(1), although occasional resizing copies the array. Inserting or removing at the front or middle requires shifting elements and is O(n).

LinkedList stores each element in a separate node containing links. Adding or removing at either end is O(1), but finding an index requires walking through nodes and is O(n). Inserting in the middle is O(1) only after the correct node/iterator position has been reached; locating that position is O(n).

ArrayDeque is a resizable circular-array deque. It is efficient for adding and removing at both ends, normally amortized O(1). It does not provide indexed get(k), so it is not a direct replacement for a playlist where frequent random access is required.

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	Amortized O(1)	O(1)	Amortized O(1)
Insert at front	O(n)	O(1)	Amortized O(1)
Insert in middle	O(n)	O(n) to locate; O(1) link change	O(n) if rebuilding/traversing
Remove at front	O(n)	O(1)	Amortized O(1)
Get k-th element	O(1)	O(n)	No indexed access; O(n) traversal
Iterate all elements	O(n), cache-friendly	O(n), pointer traversal	O(n)

1. Playlist size and tests

Use at least three sizes such as 1,000, 10,000 and 100,000 songs. For every size, measure end insertion, front insertion/removal, middle insertion/removal, indexed access and full iteration separately. Recreate or reset the list before each test so that one operation does not change the input size for the next operation.

```
import java.util.*;



public class PlaylistTest {

static long timeAddEnd(List<Integer> list, int n) { long start = System.nanoTime();

for (int i = 0; i < n; i++) list.add(i); return System.nanoTime() - start;

}



static long timeFrontInsert(List<Integer> list, int n) { long start = System.nanoTime();

for (int i = 0; i < n; i++) list.add(0, i); return System.nanoTime() - start;

}



static long timeGet(List<Integer> list, int n) { long sum = 0;
```

```
long start = System.nanoTime();

for (int i = 0; i < n; i++) sum += list.get(i); long elapsed = System.nanoTime() - start; System.out.println("checksum=" +
sum);

return elapsed;

}



static long timeIterate(List<Integer> list) { long sum = 0;

long start = System.nanoTime();

for (int value : list) sum += value;

long elapsed = System.nanoTime() - start; System.out.println("checksum=" + sum); return elapsed;

}



public static void main(String[] args) { int[] sizes = {1000, 10000, 100000};

for (int n : sizes) {

List<Integer> a = new ArrayList<>(); List<Integer> l = new LinkedList<>(); System.out.println("\\nSIZE " + n);

System.out.println("ArrayList end ns=" + timeAddEnd(a, n)); System.out.println("LinkedList end ns=" + timeAddEnd(l, n));



a = new ArrayList<>(); l = new LinkedList<>();

for (int i = 0; i < n; i++) { a.add(i); l.add(i); } System.out.println("ArrayList front insert ns=" +

timeFrontInsert(a, n / 10)); System.out.println("LinkedList front insert ns=" +

timeFrontInsert(l, n / 10)); System.out.println("ArrayList indexed get ns=" + timeGet(a, n));
System.out.println("LinkedList indexed get ns=" + timeGet(l, n)); System.out.println("ArrayList iteration ns=" +
timeIterate(a)); System.out.println("LinkedList iteration ns=" + timeIterate(l));

}

}

}
```

For a complete submission, add middle insertion/removal timings as separate methods and use a ListIterator to measure LinkedList insertion at a position already reached. Do not use repeated LinkedList.get(i) to represent normal iteration; that can turn a full indexed loop into $O(n^2)$.

1. Engineering recommendation

ArrayList is a reasonable choice when the playlist is read frequently, users jump to a particular song, iteration is common, and insertions/removals near the beginning are relatively rare. Its compact array layout also gives good cache locality and lower per-element overhead than LinkedList.

I would stop recommending ArrayList when the dominant workload repeatedly inserts/removes at the front or at both ends, and indexed access is uncommon. For the changed 'Up Next' condition, where songs are frequently added at both ends and removed from the front, ArrayDeque is a better fit. If arbitrary middle insertion is frequent, the decision depends on whether the program already has an iterator positioned at the insertion point.

1. Space comparison and plain-array alternative

ArrayList uses an array of references plus unused capacity reserved for future growth. LinkedList uses one node per element, including the stored reference and previous/next links, so its overhead per element is larger.

ArrayDeque uses an array ring buffer and some unused capacity. A plain array with manual shifting has O(1) indexed access and O(n) front/middle insertion or removal; it is less convenient because the programmer must manage capacity and shifting.

1. Evidence table

Size	Structure / operation	Run 1 (ns)	Run 2 (ns)	Run 3 (ns)	Average (ns)
1,000	ArrayList – indexed get				
1,000	LinkedList – indexed get				
10,000	ArrayList – front insert				
10,000	LinkedList – front				

	insert				
100,000	ArrayList – iteration				
100,000	LinkedList – iteration				
100,000	ArrayDeque – add/remove ends				

Q3 – Engineer the hashCode–equals Contract

1. The contract

The equals() method defines logical equality. In this problem, two Student objects represent the same student when their registration numbers are equal. The hashCode() method helps HashSet select a bucket. The Java contract requires that if a.equals(b) is true, then a.hashCode() must equal b.hashCode(). The reverse is not required: two unequal objects may have the same hash code, and equals() is then used to distinguish them.

The supplied class overrides equals() but not hashCode(). Two Student objects with the same regNo may therefore have different identity-based hash codes. HashSet can place them in different buckets and may never call equals() to compare them. This can allow duplicate logical students.

1. Three test cases and predictions

Test	Input design	Expected logical result	Reason
1 – appears correct	Add the same object reference twice	Set size 1	The same reference is recognised; also demonstrates a case that may appear to work.
2 – exposes defect	Add two different Student objects with the same regNo	Incorrect class may produce size 2	equals is true, but different hash codes can route objects to different buckets.
3 – mutation test	Insert a student, then change a field used in hashCode	Lookup/removal may fail	Object may now belong logically to a different bucket from the one where it was stored.

The third test should use a mutable field only in a separate demonstration class, because the completed assignment class below makes regNo final. Keeping equality fields immutable is the safest design.

1. Correct implementation – completed TODO

```
import java.util.HashSet; import java.util.Objects; import java.util.Set;



class Student {

private final String regNo; private final String name;



Student(String regNo, String name) { this.regNo = regNo;

this.name = name;

}



@Override

public boolean equals(Object o) { if (this == o) return true;

if (!(o instanceof Student)) return false; Student other = (Student) o;

return Objects.equals(regNo, other.regNo);

}
```

```
@Override

public int hashCode() {

return Objects.hash(regNo);

}



@Override

public String toString() {

return regNo + " " + name;

}

}



public class Attendance {

public static void main(String[] args) { Set<Student> present = new HashSet<>(); present.add(new Student("192311001",
"Arun")); present.add(new Student("192311002", "Divya")); present.add(new Student("192311001", "Arun"));

System.out.println("Unique students: " + present.size());

}

}
```

Expected output after correction: Unique students: 2. Before correction, the output may be 3 because the two separately constructed Arun objects can have different hash codes. The exact defective output is not guaranteed by the Java specification; record what your JVM actually prints.

1. How HashSet performs lookup

1. HashSet asks the object for hashCode().
2. The hash code is transformed/used to identify a bucket in the backing hash table.
3. If the bucket contains candidates, HashSet compares hash values and then uses equals() to check logical equality.
4. If an equal object is found, add() does not insert another copy; otherwise the object is stored.

If a field used by hashCode() changes after insertion, the object remains physically stored in the bucket chosen from its old hash. A later contains() or remove() calculates a bucket from the new hash and may search the wrong location. This is why fields participating in equals() and hashCode() should normally be immutable while an object is in a hash-based collection.

1. Alternative de-duplication structures

Structure	Solution	Typical scan cost	Ordering
HashSet<Student>	Store Student with consistent equals/hashCode	O(1) average; O(n) worst case	No guaranteed order
TreeSet<Student>	Comparator compares regNo; duplicate comparator result 0 is rejected	O(log n)	Sorted by regNo
HashMap<String,Student>	Use regNo as key; putIfAbsent(regNo, student)	O(1) average	No guaranteed order

```
Map<String, Student> byRegNo = new HashMap<>(); byRegNo.putIfAbsent("192311001",

new Student("192311001", "Arun"));
```

For n unique students, all three structures use O(n) storage. TreeSet has tree-node overhead; HashSet and HashMap use a hash table with bucket storage and entry/node overhead. A HashMap keyed directly by registration number is especially clear when the registration number is the unique identifier.

Q4 – Choosing a Map for a Pharmacy Inventory

1. Map comparison

The pharmacy uses a medicine code as the key and stock quantity as the value. The system needs fast code lookups, one report sorted by medicine code, and another report in insertion order.

Map	put / get	Iteration order	Best use in this problem
HashMap	O(1) average; O(n) worst case	No guaranteed order	Fast lookup when report order is not required.
TreeMap	O(log n)	Ascending key order by default	Naturally creates the code-order report.
LinkedHashMap	O(1) average	Insertion order by default	Naturally creates the order-added report.

A HashMap can produce a sorted report by copying its entries to a list and sorting them, which costs O(n log n). A HashMap cannot reproduce insertion order after the fact unless that order was stored separately. A TreeMap can produce insertion order only if an additional sequence number or separate list is maintained.

1. Java implementation

```
import java.util.*;



public class PharmacyMaps {

static Map<String, Integer> sample(String type) { Map<String, Integer> map;

if (type.equals("TreeMap")) map = new TreeMap<>();

else if (type.equals("LinkedHashMap")) map = new LinkedHashMap<>(); else map = new HashMap<>();



map.put("MED103", 40);

map.put("MED101", 25);

map.put("MED105", 12);

map.put("MED102", 60);

map.put("MED104", 18);

return map;

}



public static void main(String[] args) { Map<String, Integer> hash = sample("HashMap"); Map<String, Integer> tree =
sample("TreeMap");

Map<String, Integer> linked = sample("LinkedHashMap");



System.out.println("Stock for MED102: " + hash.get("MED102")); System.out.println("Code order: " + tree);
System.out.println("Insertion order: " + linked);



long start = System.nanoTime(); int total = 0;

for (int i = 0; i < 100000; i++) total += hash.get("MED102");

long elapsed = System.nanoTime() - start; System.out.println("Lookup checksum=" + total); System.out.println("100000
lookups ns=" + elapsed);

}

}
```

1. Experiment plan and acceptance criterion

Test 1,000, 10,000 and 100,000 medicine records. Generate unique codes in a consistent format, insert the same key/value pairs into each map, and perform the same 100,000 lookups. Use a fixed random seed if lookups are random so that every map receives the same lookup sequence.

Define an acceptable single lookup response time before testing. For this small in-memory coursework experiment, a reasonable example target is an average below 1 millisecond per lookup, but the actual target must be justified by the application and measured environment. For 100,000 lookups, report total time and calculate average time per lookup by dividing by 100,000. A hospital production service must also consider network, database, concurrency, audit and availability costs; an in-memory microbenchmark alone does not prove production readiness.

Number of medicines	Map	100,000 lookup time (ns)	Code report time (ns)	Insertion report time (ns)	Order correct?
1,000	HashMap				
1,000	TreeMap				
1,000	LinkedHashMap				
10,000	HashMap				
10,000	TreeMap				
10,000	LinkedHashMap				
100,000	HashMap				
100,000	TreeMap				
100,000	LinkedHashMap				

1. Deployment recommendation

If the application must maintain both report orders continuously, use two purpose-specific views or choose the primary structure according to the dominant operation. A practical design can keep a HashMap for fast lookup and maintain a TreeMap or sorted report view for code-order reporting; LinkedHashMap can be used when insertion order is a requirement. Maintaining multiple structures costs extra memory and update work, so every additional index should serve a real requirement.

Hashtable is generally not preferred in new Java code because it is a legacy synchronized collection, does not permit null keys or values, and has less flexible APIs than the modern Map implementations. For concurrent access, ConcurrentHashMap is generally preferred over Hashtable.

1. Sorted array alternative

An array sorted by medicine code supports binary search in O(log n). Insertion requires finding the position in O(log n) and shifting later elements in O(n), so total insertion is O(n). Producing the code-order report is O(n) because the array is already sorted. The array uses O(n) space and can be compact, but it is less suitable when records are frequently inserted or removed. HashMap provides O(1) average lookup but no order; TreeMap provides O(log n) lookup and keeps keys sorted as updates occur.

Q5 – Derive a Generic Ranking Component from a Principle (7 Marks)

1. Design decision

Team could implement Comparable<Team> if there is one natural ranking order that should apply everywhere. However, the component is intended to rank teams, players, colleges and quiz teams, and different screens may require different ranking rules. A separate Comparator is more reusable because the ordering policy is supplied from outside the ranked class.

The required order is: points descending, goal difference descending, then team name ascending. The comparator should return a negative value when the first item should appear before the second, zero when the items are equal under the ranking rule, and a positive value when the first should appear after the second.

1. Completed generic class and comparator

```
import java.util.ArrayList; import java.util.Comparator; import java.util.List;



class Team {

final String name; final int points; final int goalDiff;



Team(String name, int points, int goalDiff) { this.name = name;

this.points = points; this.goalDiff = goalDiff;

}



@Override

public String toString() {

return name + "(" + points + "," + goalDiff + ")";

}
```

```
}

class Leaderboard<T> {
private final List<T> items = new ArrayList<>(); private final Comparator<? super T> order;

Leaderboard(Comparator<? super T> order) { this.order = order;
}

void add(T item) { items.add(item);
}

List<T> ranking() {
List<T> copy = new ArrayList<>(items); copy.sort(order);
return copy;
}
}

public class LeaderboardDemo {
public static void main(String[] args) { Comparator<Team> byRank =
Comparator.comparingInt((Team t) -> t.points).reversed()
.thenComparing(
Comparator.comparingInt((Team t) -> t.goalDiff)
.reversed())
.thenComparing(t -> t.name);

Leaderboard<Team> board = new Leaderboard<>(byRank); board.add(new Team("Panthers", 9, 4));
board.add(new Team("Eagles", 9, 6));
board.add(new Team("Cobras", 6, 1));
board.add(new Team("Bisons", 9, 4));
```

```
System.out.println(board.ranking());

}

}
```

Expected output: [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]. Bisons is above Panthers because both have 9 points and goal difference 4, so the final tie-breaker is applied. 'Bisons' comes alphabetically before 'Panthers'.

1. Generics and wildcard explanation

Leaderboard<T> stores values of one chosen type. Comparator<? super T> means that the comparator can compare T values or values of a suitable superclass of T. This is flexible because a comparator written for a parent type can be reused for a child type. A method accepting List<? extends T> is useful when it needs to read items from a list of T or a subtype of T. The wildcard is read-oriented: the method should not add arbitrary T values to that source list because its exact element subtype is unknown.

1. Comparator correctness

- Consistency of sign: reversing the order of two different items reverses the comparison sign.
- Transitivity: if A ranks before B and B ranks before C, the lexicographic comparison of points, goal difference and name also ranks A before C.
- Equality under comparator: the comparator returns zero only when all three ranking keys compare equal. If unique team names are guaranteed, this represents the intended tie resolution.
- Avoid subtraction such as b.points - a.points. Integer subtraction can overflow. comparingInt(...).reversed() is safer.
- Do not ignore a tie-breaker if the specification requires it; otherwise distinct teams can compare as equal and their order may be accidental.

2. Deliberately faulty comparator test

```
// Faulty: subtraction can overflow and the comparator ignores tie-breakers. Comparator<Team> faulty =

(a, b) -> b.points - a.points;



// Correct multi-field comparator:

Comparator<Team> correct =

Comparator.comparingInt((Team t) -> t.points).reversed()

.thenComparing(

Comparator.comparingInt((Team t) -> t.goalDiff).reversed())

.thenComparing(t -> t.name);
```

Test the correct comparator with several teams tied on points, several tied on both points and goal difference, and teams with very large positive/negative integer values to expose subtraction overflow. The correct comparator should always apply the next key when the previous key ties.

1. Alternative designs and complexity

With Comparable<Team>, the ranking rule is placed inside Team as compareTo(). Sorting is still O(n log n), and the sort may use O(n) temporary space depending on the algorithm. Comparable is convenient when one natural order exists, but it is less flexible when different ranking rules are needed. With a Comparator chain, the same Team objects can be ranked in multiple ways without modifying Team.

Three stable sorts can also produce the same order if applied from the least significant key to the most significant key: sort by name ascending, then goal difference descending, then points descending. Each sort costs O(n log n), so three sorts still have O(n log n) asymptotic complexity but perform more comparisons and may allocate additional temporary storage. A single comparator chain is simpler and normally preferable. The generic Leaderboard can be reused for players or colleges by supplying a different comparator.

Q6 – Priority, Order and Fairness in a Queue

1. Why a normal FIFO queue is insufficient

A normal Queue such as LinkedList or ArrayDeque serves the earliest arrival first. The hospital requirement is different: the lowest severity number is the most critical and must be treated first. Therefore, a FIFO queue alone cannot guarantee the required priority. A PriorityQueue orders elements according to a comparator and exposes the highest-priority element at its head.

The comparator first compares severity in ascending numerical order. When severity is equal, it compares the arrival counter in ascending order. A monotonically increasing counter is assigned when a patient is admitted, so smaller counter values represent earlier arrivals.

1. Completed PriorityQueue implementation

```
import java.util.Comparator; import java.util.PriorityQueue;



class Patient {

final String name; final int severity; final long arrival;



Patient(String name, int severity, long arrival) { this.name = name;

this.severity = severity; this.arrival = arrival;

}



@Override

public String toString() {

return name + "(S" + severity + ")";

}

}



public class Triage { private long counter = 0;



private final PriorityQueue<Patient> queue = new PriorityQueue<>(

Comparator.comparingInt((Patient p) -> p.severity)
```

```
.thenComparingLong(p -> p.arrival));



void admit(String name, int severity) { if (severity < 1 || severity > 5)

throw new IllegalArgumentException("Severity must be 1 to 5"); queue.add(new Patient(name, severity, counter++));

}



Patient next() {

return queue.poll();

}



public static void main(String[] args) { Triage er = new Triage(); er.admit("Ravi", 3);

er.admit("Meena", 1);

er.admit("John", 3);

er.admit("Asha", 5);

er.admit("Kumar", 1);

er.admit("Lata", 2);

StringBuilder order = new StringBuilder(); Patient p;

while ((p = er.next()) != null)

order.append(p).append(' '); System.out.println(order.toString().trim());

}

}
```

Expected output: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5). Meena arrived before Kumar, so the arrival counter places Meena first among severity-1 patients. Ravi arrived before John, so Ravi is first among severity-3 patients.

1. Why printing PriorityQueue directly is misleading

PriorityQueue guarantees that its head is the least element according to its comparator. It does not guarantee that the entire internal array is sorted. Printing the queue directly may show the heap's internal arrangement rather than treatment order. To display treatment order safely, repeatedly poll a copy of the queue, or copy the elements into a list and sort that list with the same comparator.

1. Complexity and growth experiment

Operation	PriorityQueue	Sorted LinkedList	Five FIFO queues
Admit patient	O(log n)	O(n) to find insertion point	O(1) to append to severity queue
Serve next patient	O(log n)	O(1) remove first	O(1) inspect severity queues and remove head
Space	O(n)	O(n), with node links	O(n) plus five queue objects

Test queue sizes of 100, 1,000 and 10,000 patients, and separately measure admission and removal. Use a repeatable severity distribution and arrival sequence. For the sorted LinkedList, insertion needs to scan to the correct position. For five FIFO queues, always check severity 1 first, then 2 and so on; the first non-empty queue supplies the next patient.

Queue size	Operation	Run 1 (ns)	Run 2 (ns)	Run 3 (ns)	Average (ns)
100	Admit all				
100	Serve all				
1,000	Admit all				
1,000	Serve all				
10,000	Admit all				
10,000	Serve all				

1. Starvation and fairness

A severity-5 patient can wait indefinitely if new severity-1 to severity-4 patients keep arriving. This is called starvation. A fairness policy can increase a patient's priority as waiting time grows. For example, after every 15 minutes of waiting,

reduce the effective severity number by one level until it reaches 1. The system must periodically recalculate priorities or rebuild the priority queue because a PriorityQueue does not automatically reorder an element when a field used by its comparator changes.

An alternative is to reserve a portion of capacity for long-waiting lower-severity patients, while preserving immediate treatment for life-threatening emergencies. Any aging rule must be approved by clinical policy; software should not independently override medical triage decisions.

1. Deployment decision

A PriorityQueue can easily handle 5,000 patient records per day in terms of algorithmic scale: insertion and removal are O(log n), and O(log 5,000) is a small number of heap comparisons. However, a city hospital system must also handle concurrent admissions, persistent records, audit trails, multiple clinicians, network failures,

privacy, clinical validation and high availability. A production deployment should use a thread-safe coordination design or a central triage service, preserve the arrival sequence, and log every priority change. The data structure alone does not make the whole hospital system safe.

1. Alternative approaches

A sorted LinkedList offers O(1) removal from the head but O(n) admission because the correct position must be found. A group of five FIFO queues gives O(1) admission and O(1) service selection for a fixed five-level severity scale, provided the policy is strictly severity-first and FIFO within each level. The five-queue design is attractive for this exact fixed range; PriorityQueue is more flexible if priorities or tie-breakers become more complex.

Q7 – Diagnose, Redesign and Validate

1. Selected limitation: repeated String concatenation in ticket generation

For this design challenge, the selected solution is Q1's original ticket generator that repeatedly uses the + operator inside a loop. This is a significant limitation when the number of fields and tickets increases because each concatenation creates a new String and copies the existing prefix again.

- **Diagnose the bottleneck**
  - The main time cost is repeated copying of the ticket text as it grows.
  - The number of intermediate String objects increases with the number of fields and tickets.
  - The work grows approximately quadratically in the number of fields per ticket when field lengths are treated as constant.
  - The limitation is caused by String immutability: the existing String cannot be extended in place.

Evidence to include: the Q1 timing table for all three workloads, the average time for each implementation, and an estimated count of concatenation-result objects. The strongest evidence is the growth trend between workloads, not one isolated timing.

- **Define the design objective**

Objective: replace repeated immutable concatenation with a mutable append buffer while preserving the exact ticket contents. A measurable objective is to reduce average ticket-generation time and the number of intermediate full-ticket String objects for the 10,000-ticket, 50-field workload. The output strings must be identical for all approaches.

- **Design the revised solution**

The redesigned method creates one StringBuilder per ticket, appends each field and delimiter, and converts the completed buffer to a String once. The builder is local to the method, so concurrent ticket-generation threads do not share it.

```
String buildTicket(String[] fields) {
```

```
StringBuilder ticket = new StringBuilder(fields.length * 24); for (int i = 0; i < fields.length; i++) {

if (i > 0) ticket.append(" | "); ticket.append(fields[i]);

}

return ticket.toString();

}
```

The initial capacity is an estimate intended to reduce buffer resizing. It should be adjusted to the expected total ticket length. If the estimate is too small, the builder can grow; if it is too large, some memory is reserved but unused. The redesign does not change the ticket format or the number of final ticket strings.

Design representation

```
TicketGenerator

|

+-- buildTicket(fields)

|

+-- create local StringBuilder

+-- append field 1

+-- append delimiter and field 2

+-- ... repeat for n fields

+-- convert once with toString()

+-- return completed ticket
```

- ## Predict before implementing

Prediction: for a small ticket, the difference may be minor. As the field count and ticket count increase, StringBuilder should show a lower average execution time and fewer intermediate full String objects. The expected complexity changes from $O(n^2)$ character copying to $O(n)$ amortized append work for fixed-size fields.

- ## Implement and validate

Run the original and redesigned methods with identical arrays, ticket counts and warm-up conditions. Repeat each measurement at least five times. Verify output correctness by comparing the strings using equals().

Record execution time, approximate heap change and estimated intermediate objects. Do not claim a speed-up until your own results support it.

Calculate improvement as: ((Original average − Redesigned average) ÷ Original average) × 100. A positive result means the redesigned version was faster. If the result is negative for a tiny workload, explain that fixed setup costs and measurement noise can dominate small tasks.

- ## Defend the final design

The redesigned solution is preferred for repeated ticket construction because it avoids repeatedly rebuilding the entire prefix. It also creates fewer temporary full String objects. The trade-offs are that StringBuilder uses a mutable buffer and may resize, and a suitable initial capacity requires an estimate. For a very short one-off expression, ordinary + concatenation is simpler and may be sufficiently efficient because modern Java compilers can optimise simple concatenations. The redesign should therefore be selected based on workload evidence, not on the assumption that one class is always best.

Conclusion

The assignment demonstrates that collection and string choices should follow the operations and workload required by the application. StringBuilder is appropriate for repeated local text construction; ArrayList is strong for indexed playlist access; hashCode() must agree with equals() for correct HashSet behaviour; TreeMap and LinkedHashMap provide different ordering guarantees; Comparator and generics enable reusable ranking; and PriorityQueue supports severity-based triage. The Day-4 redesign shows the complete engineering process: diagnose a real bottleneck, define an objective, predict, implement, measure and evaluate.